Excel 활용 Big Data

『6과목-보충』
Data Analysis과 인공지능

2024.08.26-2024.08.30

Prepared by DaeKyeong Ki

Ph.D.

삼성전자 구미사업부

HS Global
HARDWARE SOFTWARE

1. SLP를 통한 게이트 문제 풀기

XOR 문제 해결

❖ 게이트 조합을 통한 XOR 문제 해결을 퍼셉트론으로 해결할 수 있을까?

✓ 풀 수 없다-선형회귀 실습

train.txt

0 0 0
0 1 1
1 0 1
1 1 0

*train.txt - Windows 메모장

파일(F) 편집(E) 서식(O) 보기(V

0 0 0
0 1 1
1 0 1
1 1 0

```
#import tensorflow as tf
import tensorflow.compat.v1 as tf
tf.disable_v2_behavior()
import numpy as np

xy = np.loadtxt('C:WWDEVWWtrain.txt')
x_data = xy[0:-1]
y_data = xy[-1]

X = tf.placeholder(tf.float32)
Y = tf.placeholder(tf.float32)

W = tf.Variable(tf.random_uniform([1, len(x_data)], -1.0, 1.0))

# Our hypothesis
h = tf.matmul(W, X)
hypothesis = tf.div(1., 1.+tf.exp(-h))

# Cross entropy cost function
cost = -tf.reduce_mean(Y*tf.log(hypothesis) + (1-Y)*tf.log(1-hypothesis))

learning_rate = tf.Variable(0.01)
optimizer = tf.train.GradientDescentOptimizer(learning_rate)
train = optimizer.minimize(cost)

init = tf.global_variables_initializer()
```

학습데이터

변수선언

가설식 정의

비용 함수

최적화 함수

주어진 변수 실행 중 바꾸기 위해 선언

- Placeholder 실행시점에 데이터를 입력하는 방법

1. SLP를 통한 게이트 문제 풀기

XOR 문제 해결

❖ 게이트 조합을 통한 XOR 문제 해결을 퍼셉트론으로 해결할 수 있을까?

✓ 풀 수 없다-선형회귀 실습

텐서플로우는 그래프를 실행하기 위해 Sesseion을 생성

최적값 찾기 반복

```
with tf.Session() as sess:
    sess.run(init)

    for step in range(5001):
        sess.run(train, feed_dict={X:x_data, Y:y_data })
        if step % 500 == 0:
            print(step, sess.run(cost, feed_dict={X:x_data, Y:y_data}), sess.run(W))

    # Test model
    correct_prediction = tf.equal(tf.floor(hypothesis+0.5), Y)
    # Calculate Accuracy
    accuracy = tf.reduce_mean(tf.cast(correct_prediction, "float"))
    print(sess.run([hypothesis, tf.floor(hypothesis+0.5), correct_prediction,
accuracy],
                   feed_dict={X:x_data, Y:y_data}))
    print("Accuracy:", accuracy.eval({X:x_data, Y:y_data}))
```

1. SLP를 통한 게이트 문제 풀기

XOR 문제 해결

```
In [41]: with tf.Session() as sess:
             sess.run(init)
             for step in range(5001):
                 sess.run(train, feed_dict={X:x_data, Y:y_data })
                 if step % 500 == 0:
                     print(step, sess.run(cost, feed_dict={X:x_data, Y:y_data}), sess.run(W))

             # Test model
             correct_prediction = tf.equal(tf.floor(hypothesis+0.5), Y)

             # Calculate Accuracy
             accuracy = tf.reduce_mean(tf.cast(correct_prediction, "float"))
             print(sess.run([hypothesis, tf.floor(hypothesis+0.5), correct_prediction, accuracy],feed_dict={X:x_data, Y:y_data}))
             print("Accuracy:", accuracy.eval({X:x_data, Y:y_data}))

         0 0.7128673 [[ 0.18365502 -0.4337554  -0.09597238]]
         500 0.6956479 [[ 0.18365502 -0.18822595  0.03516323]]
         1000 0.6937193 [[ 0.18365502 -0.09558547  0.05173251]]
         1500 0.6933535 [[ 0.18365502 -0.05483267  0.0422827 ]]
         2000 0.6932333 [[ 0.18365502 -0.0338028   0.03021215]]
         2500 0.6931844 [[ 0.18365502 -0.02161107  0.02058387]]
         3000 0.69316334 [[ 0.18365502 -0.01405295  0.01375912]]
         3500 0.69315416 [[ 0.18365502 -0.00920789  0.00912386]]
         4000 0.6931502 [[ 0.18365502 -0.0060535   0.00602946]]
         4500 0.69314843 [[ 0.18365502 -0.00398554  0.00397867]]
         5000 0.69314784 [[ 0.18365502 -0.0026257   0.00262372]]
         [array([[0.5006559 , 0.49934354, 0.49999952]], dtype=float32), array([[1., 0., 0.]], dtype=float32), array([[ True, False,
         True]]), 0.6666667]
         Accuracy: 0.6666667
```

❖Accuracy를 보면 0.67로 검토 필요성이 없음

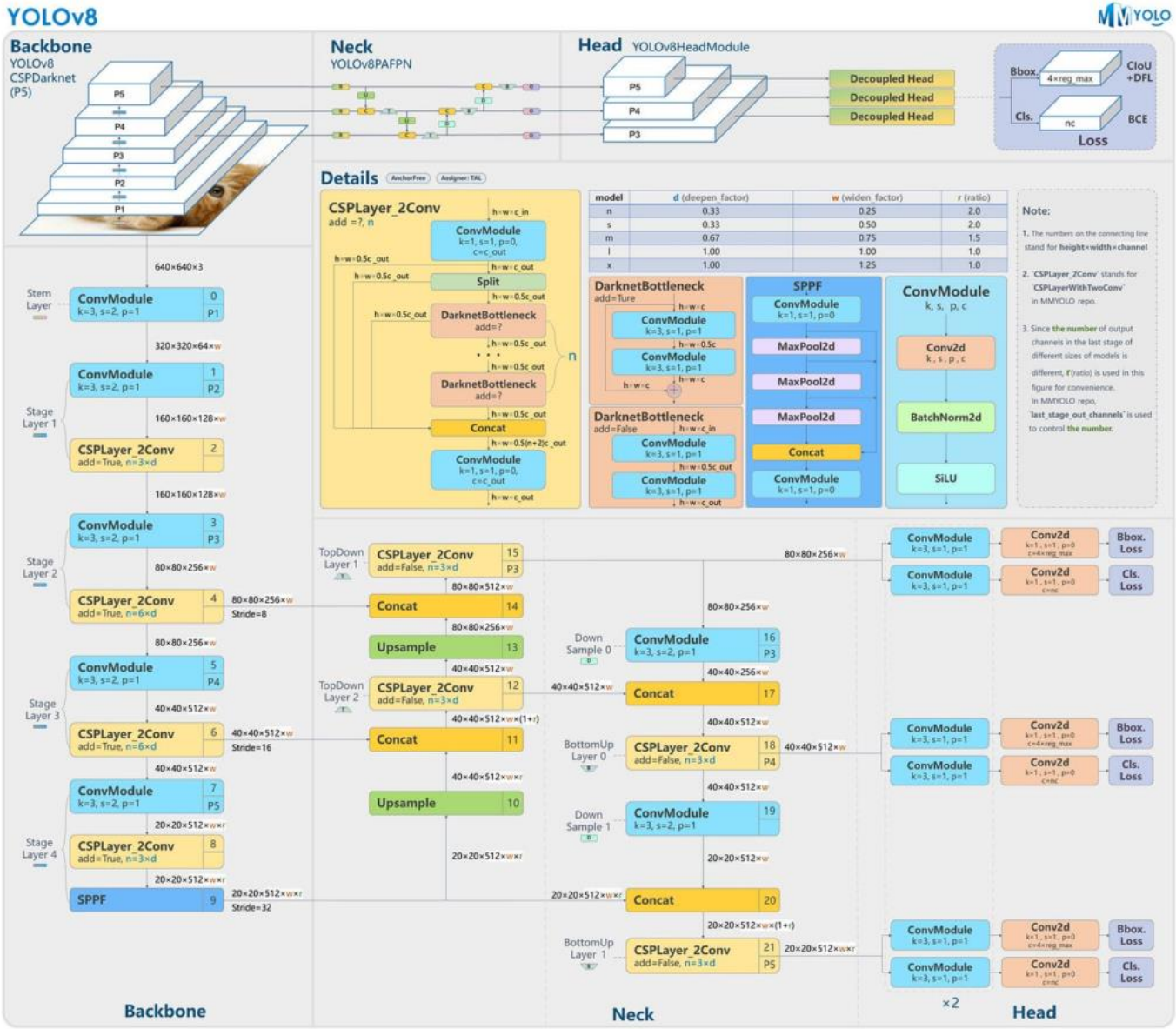
2. YOLO (You Only Look Once)

YOLOv8 이란

- YOLO 모델은 단일 단계의 객체 탐지기입니다. YOLO 모델에서 이미지 프레임은 백본(backbone)을 통해 피처화 됩니다.
- 이러한 피처는 넥에서 결합되고 혼합되며, 그런 다음 네트워크의 헤드로 전달됩니다. YOLO는 객체 주위에 경계 상자를 그릴 위치와 클래스를 예측합니다.
- 백본(Backbone)은 객체 탐지 모델에서 이미지에서 피처를 추출하는 데 사용되는 네트워크의 일부분입니다. 일반적으로, 백본은 컨볼루션(Convolution) 레이어와 풀링(Pooling) 레이어로 구성된 심층 신경망(Convolutional Neural Network, CNN)입니다.
- 백본의 역할은 입력 이미지에서 고수준의 추상적인 피처를 추출하여 객체의 특징을 표현하는 것입니다. 이 추출된 피처는 후속 단계에서 객체의 위치와 클래스를 예측하기 위해 사용됩니다. 백본은 일반적으로 이미지의 공간적인 정보를 보존하면서 이미지를 다운샘플링하여 피처의 크기를 줄이는 역할을 합니다. 이는 계산 효율성과 모델의 성능을 향상시키는 데 도움이 됩니다.
- YOLO의 경우, 다양한 버전에서 다양한 백본 아키텍처가 사용되며, 이는 모델의 정확도와 속도에 영향을 미칠 수 있습니다. 주로 ResNet, DarkNet, MobileNet 등의 백본 아키텍처가 YOLO 모델에서 많이 사용됩니다.

2. YOLO (You Only Look Once)

YOLOv8 이란



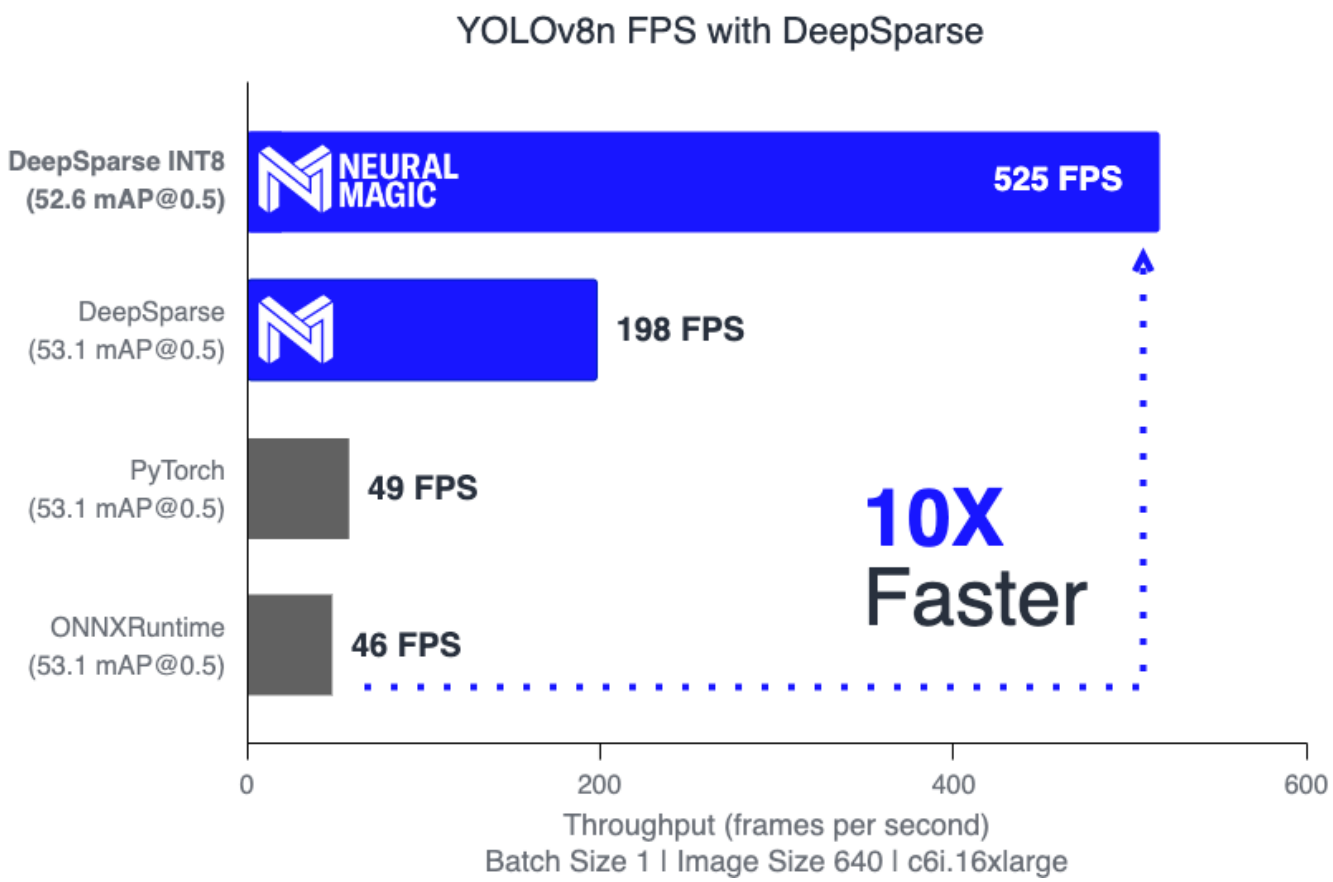
2. YOLO (You Only Look Once)

YOLOv8 이란

- YOLOv3와 YOLOv5 모델의 저자인 Ultralytics에서 개발되었으며, YOLOv8는 앵커 없는 디자인으로 객체 검출을 더욱 발전시켰습니다. 하지만 최신 정확도에만 중점을 두는 것은 아닙니다. YOLOv8는 실제 세계에서의 배포를 고려하여 속도, 지연 시간 및 가격 대비 성능에 초점을 맞췄습니다.
- YOLO의 최신 버전 및 DeepSparse를 사용하여 YOLOv8s(작은 버전)에서 209 FPS, YOLOv8n(나노 버전)에서 525 FPS를 달성하며, PyTorch 및 ONNX Runtime에 비해 10배의 가속을 얻을 수 있습니다!

2. YOLO (You Only Look Once)

YOLOv8 이란



2. YOLO (You Only Look Once)

YOLOv8를 사용하는 이유

- YOLOv8은 높은 정확도를 가지고 있으며, COCO 및 Roboflow 100에서 측정된 결과가 이를 입증하고 있습니다.

- 많은 개발자 편의 기능을 제공합니다. 사용하기 쉬운 CLI(Command Line Interface)와 잘 구성된 Python 패키지 등의 기능을 제공합니다.

2. YOLO (You Only Look Once)

적용 가이드

- 적용 가이드는 아래 링크에 자세히 나와있습니다.

- https://docs.ultralytics.com/
- https://dev.to/andreygermanov/how-to-create-yolov8-based-object-detection-web-service-using-python-julia-nodejs-javascript-go-and-rust-4o8e
- https://github.com/sjinzh/awesome-yolo-object-detection

Where to Start

- **Install** `ultralytics` with pip and get up and running in minutes — Get Started
- **Predict** new images and videos with YOLOv8 — Predict on Images
- **Train** a new YOLOv8 model on your own custom dataset — Train a Model
- **Explore** YOLOv8 tasks like segment, classify, pose and track — Explore Tasks

2. YOLO (You Only Look Once)

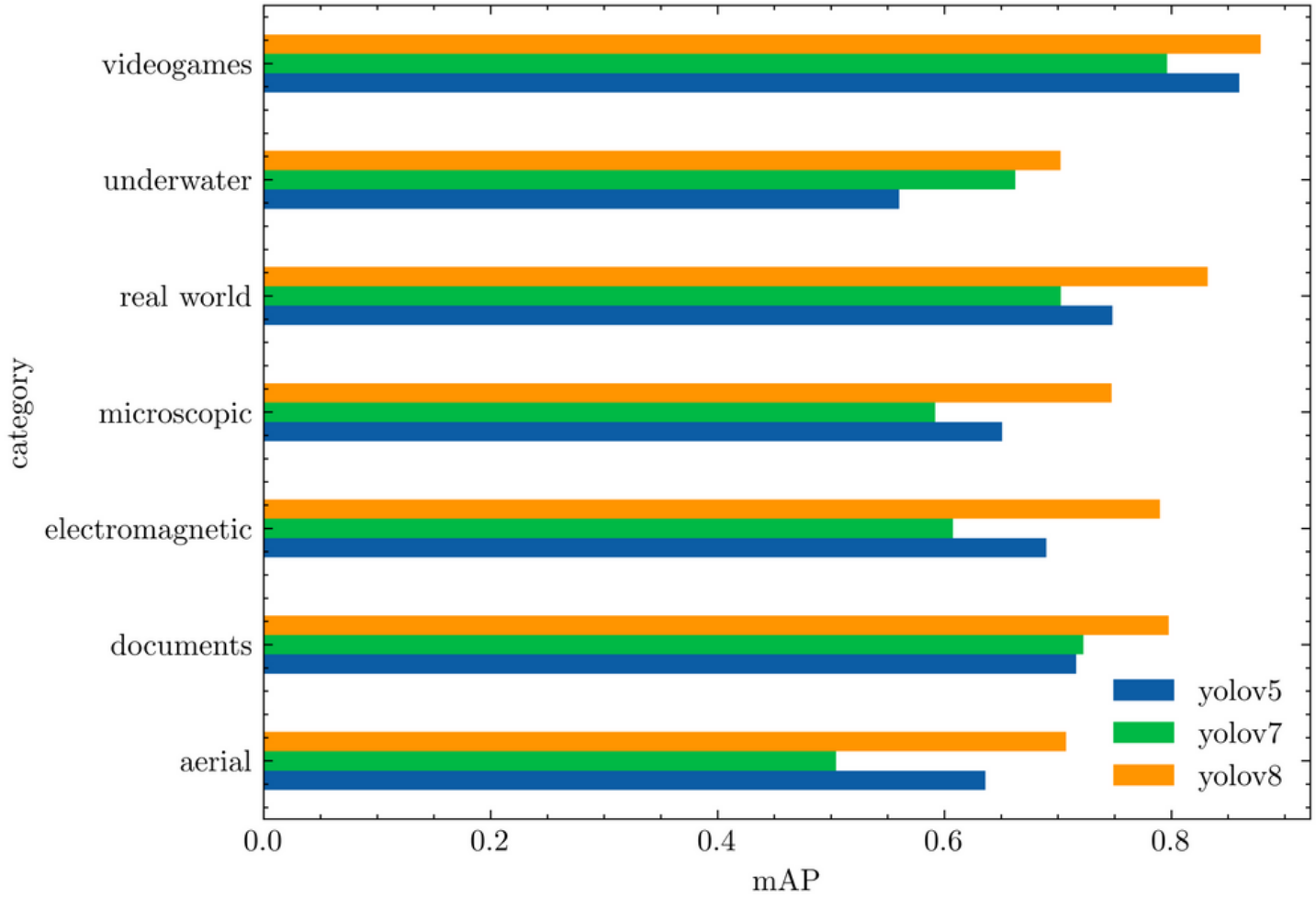
어디에 적용 할 수 있을까?

- 객체 추적은 시간에 따라 프레임 시퀀스에서 특정 객체 또는 여러 객체를 식별하는 과정입니다. 이는 각 프레임에서 객체의 위치를 파악하고 프레임 간 이동을 추적하는 것을 포함합니다. 객체 추적은 감시, 로봇공학, 자율 주행, 스포츠 분석 등 다양한 분야에서 다양한 응용 분야를 가지고 있습니다.

- 추적 알고리즘은 색상 히스토그램, 움직임 분석, 딥 러닝 등 다양한 기술을 사용하여 비디오 또는 이미지 시퀀스에서 객체를 찾고 따라가는 것입니다. 객체 감지와 추적이 다양한 산업에서 증가함에 따라 정확하고 실시간 추적 알고리즘의 필요성은 이전보다 더욱 중요해졌습니다.
- 객체를 정확하고 신속하게 추적하는 능력은 보안 감시와 같은 응용 분야에서 매우 중요합니다. 이를 통해 잠재적인 위협을 식별하고 수상한 행동을 탐지하는 데 도움을 줄 수 있습니다. 스포츠 분석 분야에서는 객체 추적을 통해 경기 중 선수들의 움직임을 분석하여 선수의 성과와 게임 전략에 대한 유용한 통찰력을 제공할 수 있습니다.

2. YOLO (You Only Look Once)

어디에 적용 할 수 있을까?



2. YOLO (You Only Look Once)

YOLO Licenses

- AGPL-3.0 License
- AGPL-3.0 라이선스는 상업적 이용을 허용합니다. 상업적으로 소프트웨어를 이용하고 수정하여 사용할 수 있습니다. 하지만 AGPL-3.0 라이선스의 조건을 준수해야 합니다.
- 이는 소스 코드의 공개, 라이선스 조건의 준수, 웹 애플리케이션의 경우 사용자에게 소스 코드 접근을 허용하는 등의 규정을 포함합니다.

- 따라서, AGPL-3.0 라이선스를 사용하는 소프트웨어를 상업적으로 이용하려면 해당 라이선스의 조건을 충족시켜야 합니다.

- 소스 코드의 공개와 라이선스 조건의 준수는 AGPL-3.0 라이선스의 핵심 요소이며, 이를 준수하지 않는다면 라이선스 위반으로 간주될 수 있습니다.

- 따라서, AGPL-3.0 라이선스를 사용하는 소프트웨어를 상업적으로 이용하고자 할 경우, 해당 라이선스의 조건을 자세히 검토하고 준수해야 합니다

YOLOv8 설치

```
c:\DEV>cd envs

c:\DEV>python -m venv py3_10_yolov8

c:\DEV\envs> cd py3_10_yolov8

c:\DEV\envs\py3_10_yolov8>Scripts\activate

(py3_10_yolov8) c:\DEV\envs\py3_10_yolov8>pip install ultralytics
Collecting ultralytics
  Downloading ultralytics-8.2.41-py3-none-any.whl (792 kB)
     ---------------------------------------- 793.0/793.0 kB 16.7 MB/s eta 0:00:00
Collecting py-cpuinfo
  Downloading py_cpuinfo-9.0.0-py3-none-any.whl (22 kB)
Collecting ultralytics-thop>=2.0.0

…
```

YOLOv8 설치

```
(py3_10_yolov8) c:\DEV\envs\py3_10_yolov8>pip install ipykernel
Collecting ipykernel
  Using cached ipykernel-6.29.4-py3-none-any.whl (117 kB)
Requirement already satisfied: psutil in c:\dev\envs\py3_10_yolov8\lib\site-packages (from ipykernel) (6.0.0)
Collecting ipython>=7.23.1
```

```
(py3_10_yolov8) c:\DEV\envs\py3_10_yolov8>python.exe -m pip install --upgrade pip
Requirement already satisfied: pip in c:\dev\envs\py3_10_yolov8\lib\site-packages (23.0.1)
Collecting pip
  Using cached pip-24.1-py3-none-any.whl (1.8 MB)
…23.0.1
    Uninstalling pip-23.0.1:
      Successfully uninstalled pip-23.0.1
Successfully installed pip-24.1
```

```
(py3_10_yolov8) c:\DEV\envs\py3_10_yolov8>python -m ipykernel install --user --name
py3_10_yolov8 --display-name "py3_10_yolov8"
Installed kernelspec py3_10_yolov8 in C:\Users\k8s\AppData\Roaming\jupyter\kernels\py3_10_yolov8
```

```
(py3_10_yolov8) c:\DEV\envs\py3_10_yolov8>deactivate
c:\DEV\envs\py3_10_yolov8>cd ..

c:\DEV\envs>cd ..

c:\DEV>jupyter notebook
```

2. YOLO (You Only Look Once)

공식 홈페이지에 안내하는 코드

```
from ultralytics import YOLO

# Load a model
model = YOLO('yolov8n.pt')  # pretrained YOLOv8n model

# Run batched inference on a list of images
results = model(['im1.jpg', 'im2.jpg'])  # return a list of Results objects

# Process results list
for result in results:
    boxes = result.boxes  # Boxes object for bbox outputs
    masks = result.masks  # Masks object for segmentation masks outputs
    keypoints = result.keypoints  # Keypoints object for pose outputs
    probs = result.probs  # Class probabilities for classification outputs
```

❖ 경로 문제로 에러남

2. YOLO (You Only Look Once)

공식 홈페이지에 안내하는 코드

C:\aabb\serch\yolotest

2. YOLO (You Only Look Once)

공식 홈페이지에 안내하는 코드

```
from ultralytics import YOLO

# Load a model
model = YOLO('yolov8n.pt')  # pretrained YOLOv8n model

# Define image file path
image_folder = 'C:\\aabb\\serch\\yolotest\\'

# Define image file names
image_file_1 = 'im1.jpg'
image_file_2 = 'im2.jpg'

# Construct full image file paths
image_path_1 = image_folder + image_file_1
image_path_2 = image_folder + image_file_2

# Run batched inference on a list of images
results = model([image_path_1, image_path_2])  # return a list of Results objects

# Process results list
for result in results:
    boxes = result.boxes  # Boxes object for bbox outputs
    masks = result.masks  # Masks object for segmentation masks outputs
    keypoints = result.keypoints  # Keypoints object for pose outputs
    probs = result.probs  # Class probabilities for classification outputs
```

2. YOLO (You Only Look Once)

공식 홈페이지에 안내하는 코드

```
Downloading https://github.com/ultralytics/assets/releases/download/v8.2.0/yolov8n.pt to 'yolov8n.pt'...
100%|██████████████████████████████████████████████████████████████████████████████| 6.23M/6.23M [00:00<00:00, 103MB/s]

0: 256x640 15 persons, 1 car, 3 buss, 171.5ms
1: 256x640 15 persons, 1 car, 3 buss, 171.5ms
Speed: 2.5ms preprocess, 171.5ms inference, 4.6ms postprocess per image at shape (1, 3, 256, 640)
```

1. pretained모델 yolov8n.pt파일을 불러오는데 여기서 이 용어는

2. "pretrained"는 미리 학습된 모델을 의미합니다. 딥 러닝 모델은 대량의 데이터셋에서 학습하여 일반적인 이미지 인식, 객체 감지, 자연어 처리 등의 작업을 수행할 수 있도록 학습됩니다. 하지만 이러한 모델을 처음부터 학습 시키는 데에는 많은 시간과 계산 리소스가 필요합니다.

3. 따라서, pretrained 모델은 이미 대규모 데이터셋에서 사전에 학습되어 일반적인 패턴과 특징을 학습한 모델을 의미합니다. 이러한 모델을 사용하면 처음부터 학습시키는 대신, 이전에 학습된 모델을 기반으로 새로운 작업에 대한 예측을 수행할 수 있습니다. pretrained 모델은 일반적으로 이미지넷(ImageNet)과 같은 대형 데이터셋에서 사전 학습된 모델을 의미합니다. 이 모델들은 이미지 분류, 객체 감지, 세그멘테이션 등 다양한 컴퓨터 비전 작업에 사용될 수 있습니다. pretrained 모델을 사용하면 소규모 데이터셋에서도 좋은 성능을 얻을 수 있고, 학습 시간과 비용을 절약할 수 있습니다.

2. YOLO (You Only Look Once)

공식 홈페이지에 안내하는 코드

참고로 이미지 입력 방법

Source	Argument	Type	Notes
image	`'image.jpg'`	`str` or `Path`	Single image file.
URL	`'https://ultralytics.com/images/bus.jpg'`	`str`	URL to an image.
screenshot	`'screen'`	`str`	Capture a screenshot.
PIL	`Image.open('im.jpg')`	`PIL.Image`	HWC format with RGB channels.
OpenCV	`cv2.imread('im.jpg')`	`np.ndarray` of `uint8 (0-255)`	HWC format with BGR channels.
numpy	`np.zeros((640,1280,3))`	`np.ndarray` of `uint8 (0-255)`	HWC format with BGR channels.
torch	`torch.zeros(16,3,320,640)`	`torch.Tensor` of `float32 (0.0-1.0)`	BCHW format with RGB channels.
CSV	`'sources.csv'`	`str` or `Path`	CSV file containing paths to images, videos, or directories.
video ✅	`'video.mp4'`	`str` or `Path`	Video file in formats like MP4, AVI, etc.
directory ✅	`'path/'`	`str` or `Path`	Path to a directory containing images or videos.
glob ✅	`'path/*.jpg'`	`str`	Glob pattern to match multiple files. Use the `*` character as a wildcard.
YouTube ✅	`'https://youtu.be/Zgi9g1ksQHc'`	`str`	URL to a YouTube video.
stream ✅	`'rtsp://example.com/media.mp4'`	`str`	URL for streaming protocols such as RTSP, RTMP, or an IP address.

2. YOLO (You Only Look Once)

박스형상 으로 확인

```
from ultralytics import YOLO
import cv2
# Load a model
model = YOLO('yolov8n.pt')  # pretrained YOLOv8n model
model.predict(source="0", show=True, stream=True, classes=0)  # [0, 3, 5] for multiple classes


# Define image file path
image_folder = 'C:\\aabb\\serch\\yolotest\\'

# Define image file names
image_file_1 = 'im1.jpg'
image_file_2 = 'im2.jpg'

# Construct full image file paths
image_path_1 = image_folder + image_file_1
image_path_2 = image_folder + image_file_2

# Run batched inference on a list of images
results = model([image_path_1, image_path_2])  # return a list of Results objects
model.predict(source="0", show=True, stream=True, classes=0)

# Process results list
for result in results:
    boxes = result.boxes  # Boxes object for bbox outputs
    masks = result.masks  # Masks object for segmentation masks outputs
    keypoints = result.keypoints  # Keypoints object for pose outputs
    probs = result.probs  # Class probabilities for classification outputs

    # Print box coordinates
    print("Box coordinates:")
    for box in boxes:
        print(box)

    # Display result with boxes
    result.plot()
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

2. YOLO (You Only Look Once)

박스형상 으로 확인

1. 모델 로드
```
model = YOLO('yolov8n.pt')  # pretrained YOLOv8n model
```
YOLO('yolov8n.pt'): 사전 학습된 YOLOv8n 모델을 로드합니다.
2. 실시간 비디오 스트림에서 객체 감지
```
model.predict(source="0", show=True, stream=True, classes=0)
```
source="0": 기본 웹캠에서 비디오 스트림을 입력으로 받습니다.
show=True: 객체 감지 결과를 실시간으로 화면에 표시합니다.
stream=True: 비디오 스트림을 처리합니다.
classes=0: 클래스 ID 0 (예: 사람)만 감지합니다.
3. 이미지 파일 경로 설정 및 객체 감지
```
image_folder = 'C:\\aabb\\serch\\yolotest\\'

image_file_1 = 'im1.jpg'
image_file_2 = 'im2.jpg'

image_path_1 = image_folder + image_file_1
image_path_2 = image_folder + image_file_2
```
image_folder: 이미지 파일이 저장된 폴더 경로를 지정합니다.
image_file_1, image_file_2: 이미지 파일 이름을 지정합니다.
image_path_1, image_path_2: 폴더 경로와 파일 이름을 결합하여 전체 이미지 파일 경로를 생성합니다.
4. 이미지 리스트에 대한 배치 추론
```
results = model([image_path_1, image_path_2])  # return a list of Results objects
```
model([image_path_1, image_path_2]): 이미지 파일 리스트에 대해 YOLO 모델을 사용하여 객체 감지를 수행합니다.
결과는 Results 객체의 리스트로 반환됩니다.

박스형상 으로 확인

```
5. 결과 처리 및 출력
for result in results:
    boxes = result.boxes  # Boxes object for bbox outputs
    masks = result.masks  # Masks object for segmentation masks outputs
    keypoints = result.keypoints  # Keypoints object for pose outputs
    probs = result.probs  # Class probabilities for classification outputs

    # Print box coordinates
    print("Box coordinates:")
    for box in boxes:
        print(box)

    # Display result with boxes
    result.plot()
    cv2.waitKey(0)
    cv2.destroyAllWindows()
for result in results: 각 이미지에 대한 결과를 반복합니다.
result.boxes: 바운딩 박스 객체를 가져옵니다.
result.masks: 세그멘테이션 마스크 객체를 가져옵니다.
result.keypoints: 포즈 추정 키포인트 객체를 가져옵니다.
result.probs: 분류 출력에 대한 클래스 확률을 가져옵니다.
print(box): 바운딩 박스 좌표를 출력합니다.
result.plot(): 결과 이미지에 바운딩 박스를 그려서 표시합니다.
cv2.waitKey(0), cv2.destroyAllWindows(): 결과 이미지를 보여준 후 키 입력을 기다리고 모든 창을 닫습니다.
```

2. YOLO (You Only Look Once)

박스형상 으로 확인

View Run Kernel Settings Help

Code

```
ultralytics.engine.results.Boxes ob

cls: tensor([0.])
conf: tensor([0.2702])
data: tensor([[9.1430e+01, 1.6962e+
id: None
is_track: False
orig_shape: (391, 1024)
shape: torch.Size([1, 6])
xywh: tensor([[109.3379, 241.3658,
xywhn: tensor([[0.1068, 0.6173, 0.0
xyxy: tensor([[ 91.4296, 169.6183,
xyxyn: tensor([[0.0893, 0.4338, 0.1
```

2. YOLO (You Only Look Once)

특정 사물만 인식

YOLOv8 특정 사물만 인식하는 방법은 아주 간단합니다.

classes=0 이 부분에 숫자가 인식하고자 하는 형상 번호 입니다.

```
model.predict(source="0", show=True, stream=True, classes=0)  # [0, 3, 5] for multiple classes
```

2. YOLO (You Only Look Once)

특정 사물만 인식

```
from ultralytics import YOLO
import cv2
# Load a model
model = YOLO('yolov8n.pt')  # pretrained YOLOv8n model
model.predict(source="0", show=True, stream=True, classes=0)  # [0, 3, 5] for multiple classes

# Define image file path
image_folder = 'C:\\aabb\\serch\\yolotest\\'

# Define image file names
image_file_1 = 'im1.jpg'
image_file_2 = 'im2.jpg'

# Construct full image file paths
image_path_1 = image_folder + image_file_1
image_path_2 = image_folder + image_file_2

# Run batched inference on a list of images
results = model([image_path_1, image_path_2])  # return a list of Results objects

# Process results list
for result in results:
    boxes = result.boxes  # Boxes object for bbox outputs
    masks = result.masks  # Masks object for segmentation masks outputs
    keypoints = result.keypoints  # Keypoints object for pose outputs
    probs = result.probs  # Class probabilities for classification outputs

    # Print box coordinates
    print("Box coordinates:")
    for box in boxes:
        print(box)

    # Display result with boxes
    result.plot()
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

2. YOLO (You Only Look Once)

특정 사물만 인식

코드를 실행하면 이전에 차량과 사람을 모두 인식했던 결과와 다르게 사람만 인식한 결과를 확인할 수 있습니다.

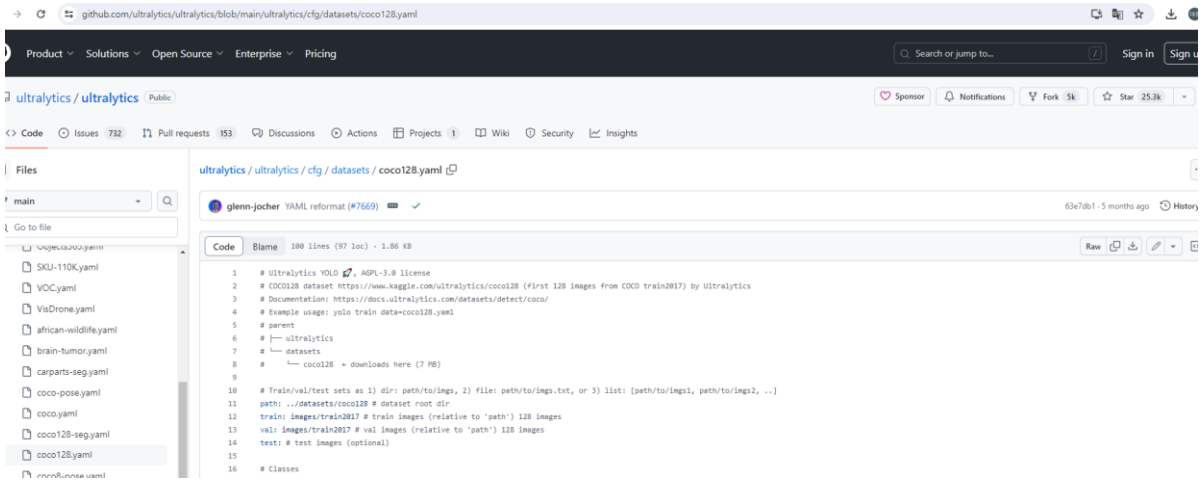
2. YOLO (You Only Look Once)

선행된(pre-trained) YOLOv8 형상 클래스

YOLOv8 특정 사물만 인식하는 방법에 필요한 클래스(Class)는 아래의 80가지 클래스로 학습이 되어있습니다.

```
names:
0: person
1: bicycle
2: car
3: motorcycle
4: airplane
5: bus
6: train
7: truck
8: boat
9: traffic light
10: fire hydrant
11: stop sign
12: parking meter
13: bench
14: bird
15: cat
16: dog
17: horse
18: sheep
19: cow
20: elephant
21: bear
22: zebra
23: giraffe
24: backpack
25: umbrella
26: handbag
27: tie
28: suitcase
29: frisbee
30: skis
31: snowboard
32: sports ball
33: kite
34: baseball bat
35: baseball glove
```

위 정보가 있는 coco128.yaml은 여기에 있습니다.



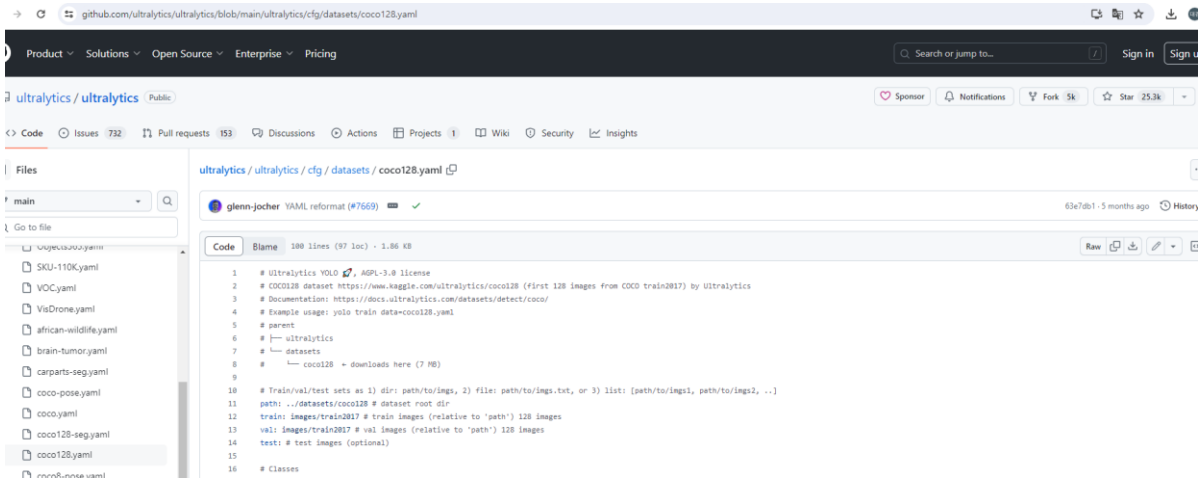
2. YOLO (You Only Look Once)

선행된(pre-trained) YOLOv8 형상 클래스

YOLOv8 특정 사물만 인식하는 방법에 필요한 클래스(Class)는 아래의 80가지 클래스로 학습이 되어있습니다.

```
36: skateboard
37: surfboard
38: tennis racket
39: bottle
40: wine glass
41: cup
42: fork
43: knife
44: spoon
45: bowl
46: banana
47: apple
48: sandwich
49: orange
50: broccoli
51: carrot
52: hot dog
53: pizza
54: donut
55: cake
56: chair
57: couch
58: potted plant
59: bed
60: dining table
61: toilet
62: tv
63: laptop
64: mouse
65: remote
66: keyboard
67: cell phone
68: microwave
69: oven
70: toaster
71: sink
72: refrigerator
73: book
74: clock
75: vase
76: scissors
77: teddy bear
78: hair drier
79: toothbrush
```

위 정보가 있는 coco128.yaml은 여기에 있습니다.



2. YOLO (You Only Look Once)

1. 개요

- YOLO를 통해 custom image를 학습시키는 경우 데이터셋이 필요하다.
- 자신이 원하는 데이터셋이 존재하지 않을 경우 직접 데이터셋을 만들어줘야 하는데, Image에서 Bounding Box를 지정하여 라벨링을 수행하고 Bounding Box 정보들을 좌표파일로 생성해야 한다.
- 이미지 라벨링 툴 labelImg를 이용해서 이미지 라벨링을 수행할 수 있다.

2. YOLO (You Only Look Once)

2. 설치 및 실행

- https://github.com/heartexlabs/labelImg
- https://github.com/tzutalin/labelImg
- 위 링크에 접속하면 다음과 같은 화면을 볼 수 있을 것이다.
- 그 중에서 Releases를 클릭해주자.

2. YOLO (You Only Look Once)

2. 설치 및 실행

heartexlabs/labelImg: LabelImg

github.com/heartexlabs/labelImg

도커수업 참고 클라우드 기관 도서 AI DEV 교양 쿠팡! 옥션 - 모바일 쇼핑... Openstack

Search or jump to... / Pull requests Issues Codespaces Marketplace Explore

heartexlabs / labelImg Public

Watch 377 Fork 5.7k Star 18.8k

Code Issues 362 Pull requests 43 Actions Projects Wiki Security Insights

master 5 branches 25 tags Go to file Add file Code

lsell Readme updates (#950) ... b33f965 on Sep 23 449 commits

.github	Revert "Create pylint.yml"	5 months ago
build-tools	Fix typo	4 years ago
data	Modified the default label text box into a drop down (#824)	10 months ago
demo	Screenshot of macOS High Sierra usage added.	5 years ago
libs	CreateML fixes (#906)	5 months ago
readme	Readme updates (#950)	2 months ago
requirements	Bump lxml from 4.6.5 to 4.9.1 in /requirements (#909)	5 months ago
resources	Updated translations	5 months ago
tests	CreateML fixes (#906)	5 months ago
tools	move convert dir to tools dir	2 years ago
.gitignore	Update .gitignore	5 months ago
CONTRIBUTING.rst	Create new files for pypackage	6 years ago
HISTORY.rst	bump version to 1.8.6	14 months ago
LICENSE	Create new files for pypackage	6 years ago
MANIFEST.in	Add missing files	6 years ago
Makefile	Update README.md and setup.py	4 years ago

About

LabelImg is now part of the Label Studio community. The popular image annotation tool created by Tzutalin is no longer actively being developed, but you can check out Label Studio, the open source data labeling tool for images, text, hypertext, audio, video and time-series data.

youtu.be/p0nR2YsCY_U

recognition tools deep-learning detection annotations python3 imagenet image-classification python2

Readme
MIT license
18.8k stars
377 watching
5.7k forks

Releases 2

Binary v1.8.1 Latest
on Dec 3, 2018

+ 1 release

labelImg-master.zip 모두 표시

2. YOLO (You Only Look Once)

2. 설치 및 실행

Release Binary v1.8.1 · heartexla

github.com/heartexlabs/labelImg/releases/tag/v1.8.1

도커수업 참고 클라우드 기관 도서 AI DEV 교양 쿠팡! 옥션 - 모바일 쇼핑... Openstack

Search or jump to... / Pull requests Issues Codespaces Marketplace Explore

heartexlabs / labelImg Public

Watch 377 Fork 5.7k Star 18.8k

Code Issues 362 Pull requests 43 Actions Projects Wiki Security Insights

Releases / v1.8.1

Compare

Binary v1.8.1 Latest

tzutalin released this Dec 3, 2018 · 142 commits to master since this release v1.8.1 3f14cbf

windows_v1.8.1.zip

▾ Assets 2

Source code (zip)	Dec 3, 2018
Source code (tar.gz)	Dec 3, 2018

49 3 52 people reacted

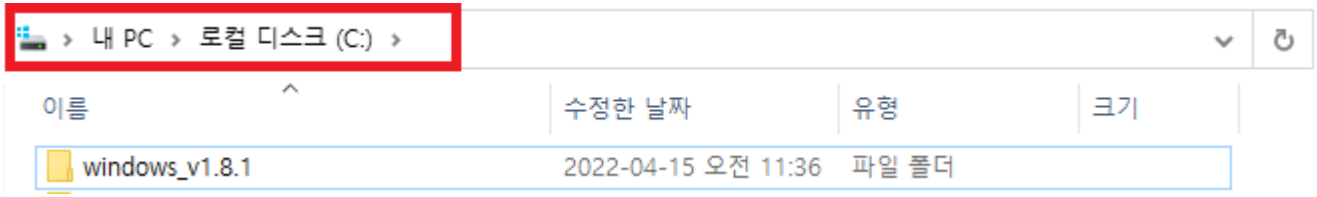
© 2022 GitHub, Inc. Terms Privacy Security Status Docs Contact GitHub Pricing API Training Blog About

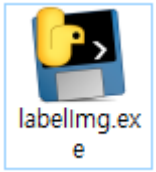
labelImg-master.zip

모두 표시

2. 설치 및 실행

- 여러가지 버전이 표시될텐데, Windows 환경이므로 Windows_v1.8.1.zip을 다운로드 해준 뒤, 압축을 해제한다.

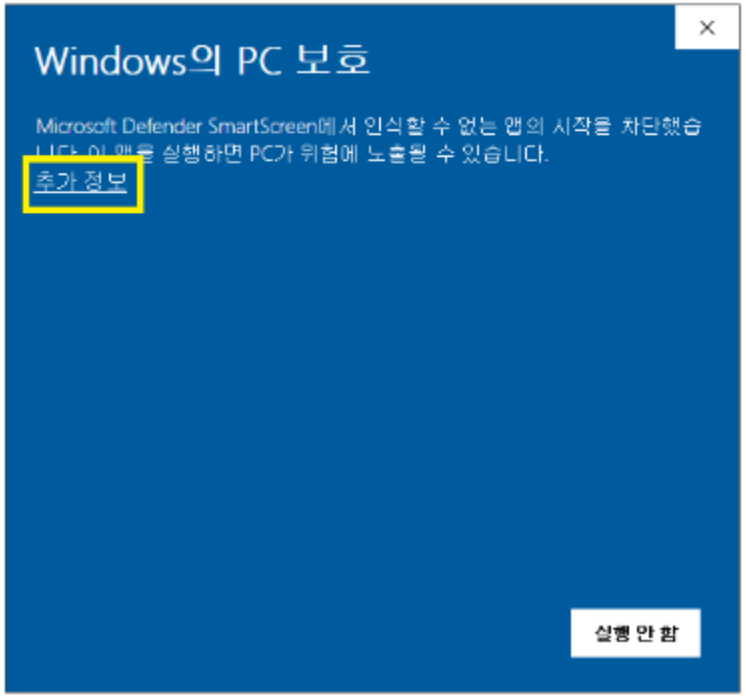


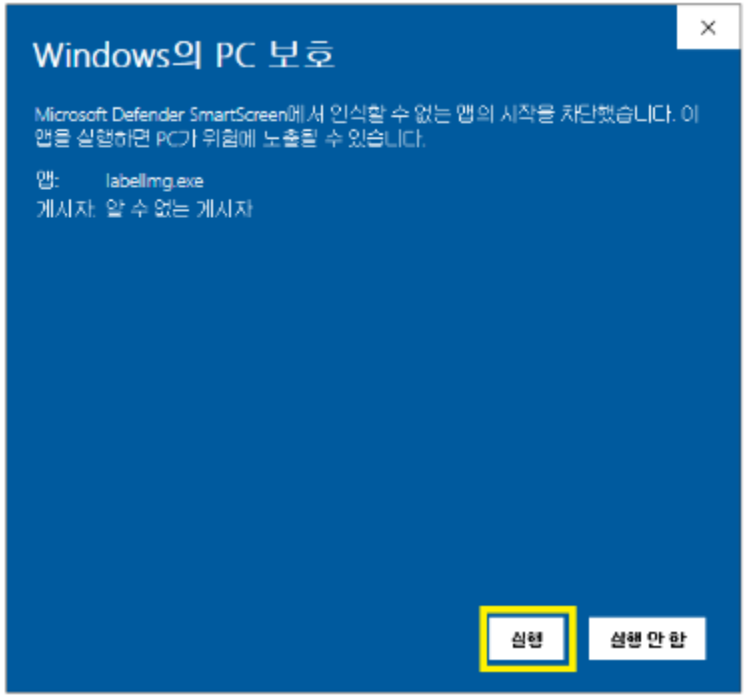


- labelImg.exe 파일을 실행시켜보자.

2. 설치 및 실행

- 추가 정보 클릭 후 실행하면 된다.



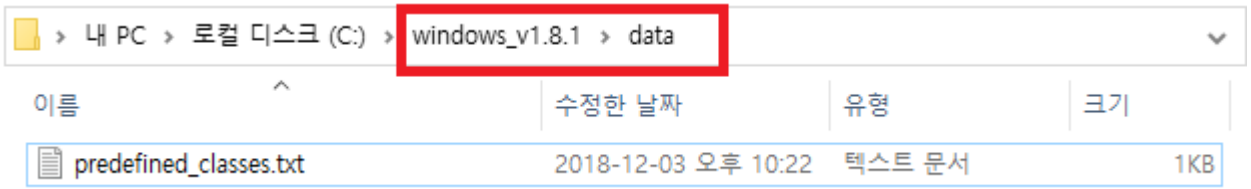


2. YOLO (You Only Look Once)

1. Class 정의

- 라벨링하고자 하는 class를 정의해주는 것과, YOLOv5의 경우 xml 이 아닌 txt 파일 형식이 필요한데, labelImg의 기본값이 xml이기 때문에 이를 변경해주고 작업해야 한다.

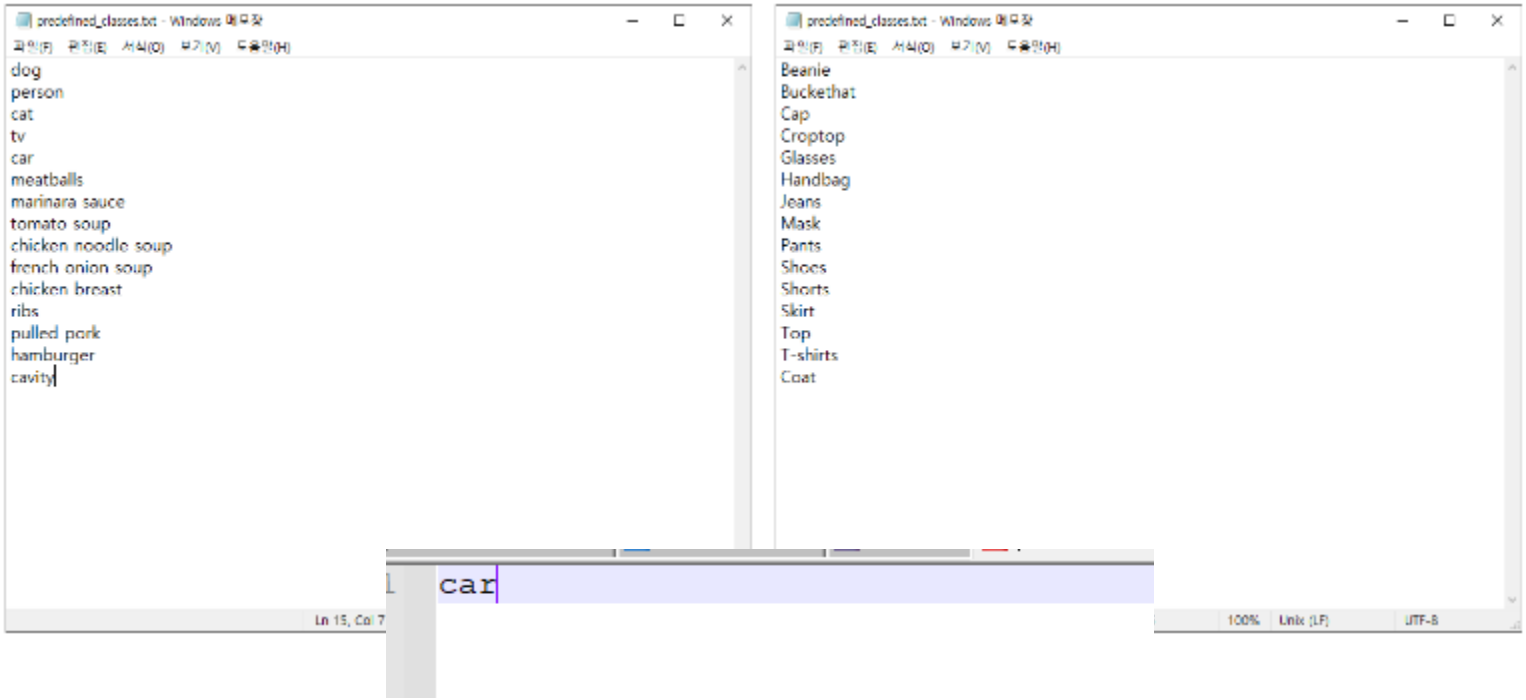
- Class 정의



2. YOLO (You Only Look Once)

1. Class 정의

- labelImg를 위치한 디렉토리 아래에 Data 디렉토리로 들어가면 predefined_classes.txt 라는 파일이 있다. 해당 파일을 편집기를 통해 열어주자. 텍스트 파일을 열어보면 다음과 같이 class들이 정의 되어있는데 이를 내가 원하는 class들로 변경해주면 된다.



2. YOLO (You Only Look Once)

2. 저장 방식 변경

- 라벨링을 진행할 때 저장되는 파일의 형식을 변경해줘야 한다. PascalVOC를 눌러 YOLO로 변경해주자. labelImg를 실행할 때마다 변경해주어야 한다.

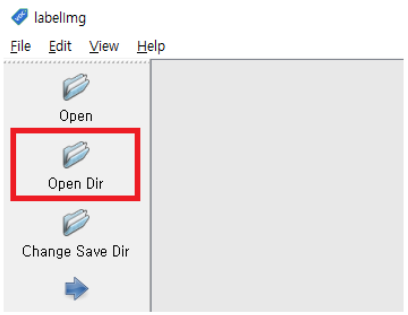
2. YOLO (You Only Look Once)

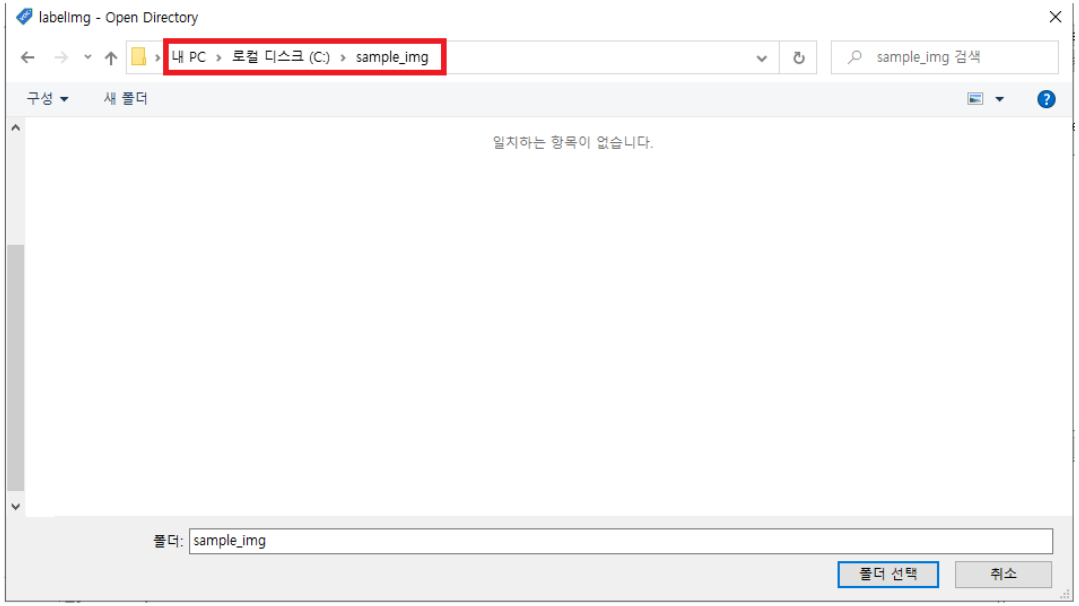
2. 저장 방식 변경

- 라벨링을 진행할 때 저장되는 파일의 형식을 변경해줘야 한다. PascalVOC를 눌러 YOLO로 변경해주자. labelImg를 실행할 때마다 변경해주어야 한다.

3. 이미지 불러오기

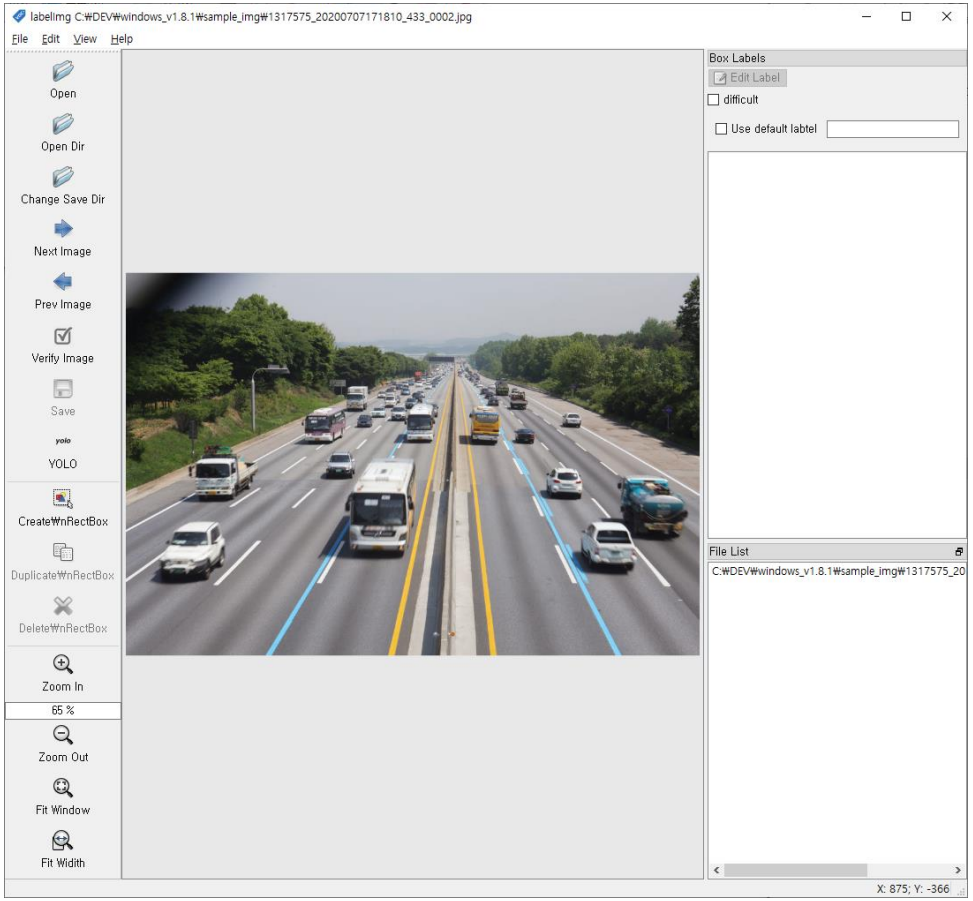
- Open Dir을 통해 이미지가 저장된 폴더를 선택해주자.





3. 이미지 불러오기

- Open Dir을 통해 이미지가 저장된 폴더를 선택해주자.



2. YOLO (You Only Look Once)

4. 단축키

- 유용한 단축키

Hotkeys

Ctrl + u	Load all of the images from a directory
Ctrl + r	Change the default annotation target dir
Ctrl + s	Save
Ctrl + d	Copy the current label and rect box
Ctrl + Shift + d	Delete the current image
Space	Flag the current image as verified
w	Create a rect box
d	Next image
a	Previous image
del	Delete the selected rect box
Ctrl++	Zoom in
Ctrl--	Zoom out
↑→↓←	Keyboard arrows to move selected rect box

2. YOLO (You Only Look Once)

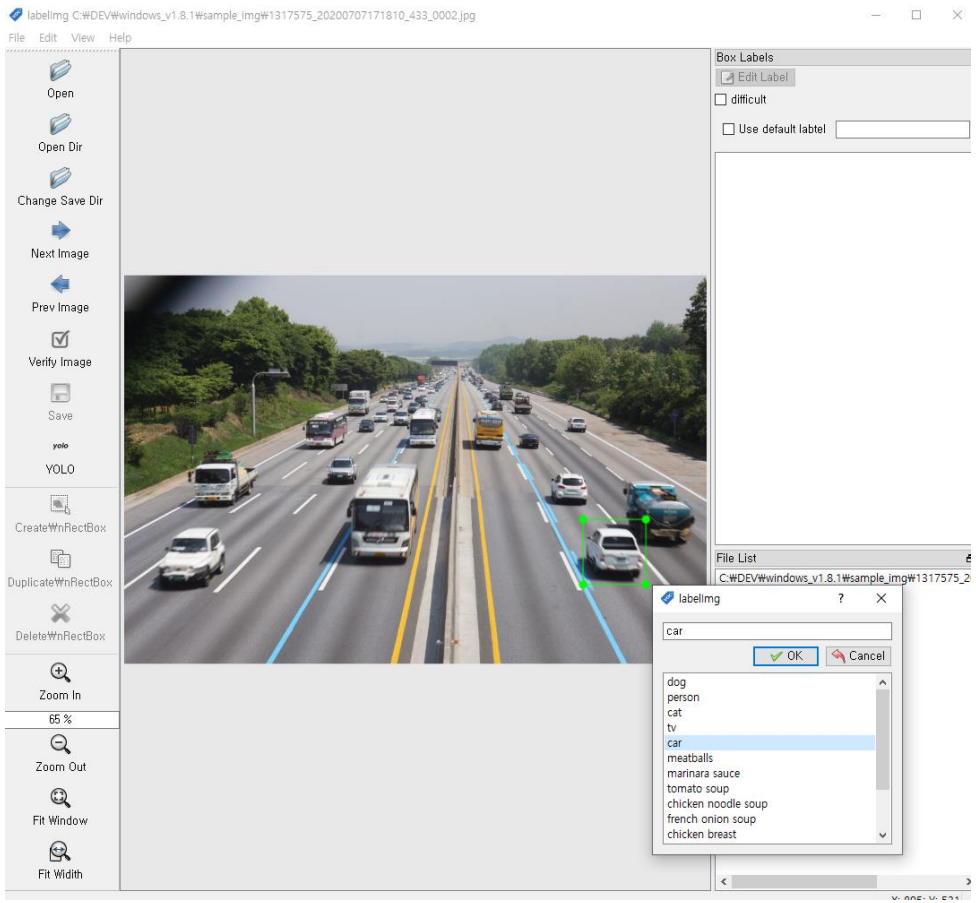
4. 단축키

- 라벨링을 진행해보자. W를 누른 후 시작점부터 끝점까지 드래그하여 Bounding Box의 영역을 설정해주자.드래그를 종료하면 다음과 같이 class를 선택하는 창이 나오는데, 해당하는 class를 선택해주면 된다. 상단의 검색 기능을 포함하고 있으므로 영역을 지정 후 해당하는 class의 첫 글자를 입력하여 빠르게 class를 찾을 수 있다.

2. YOLO (You Only Look Once)

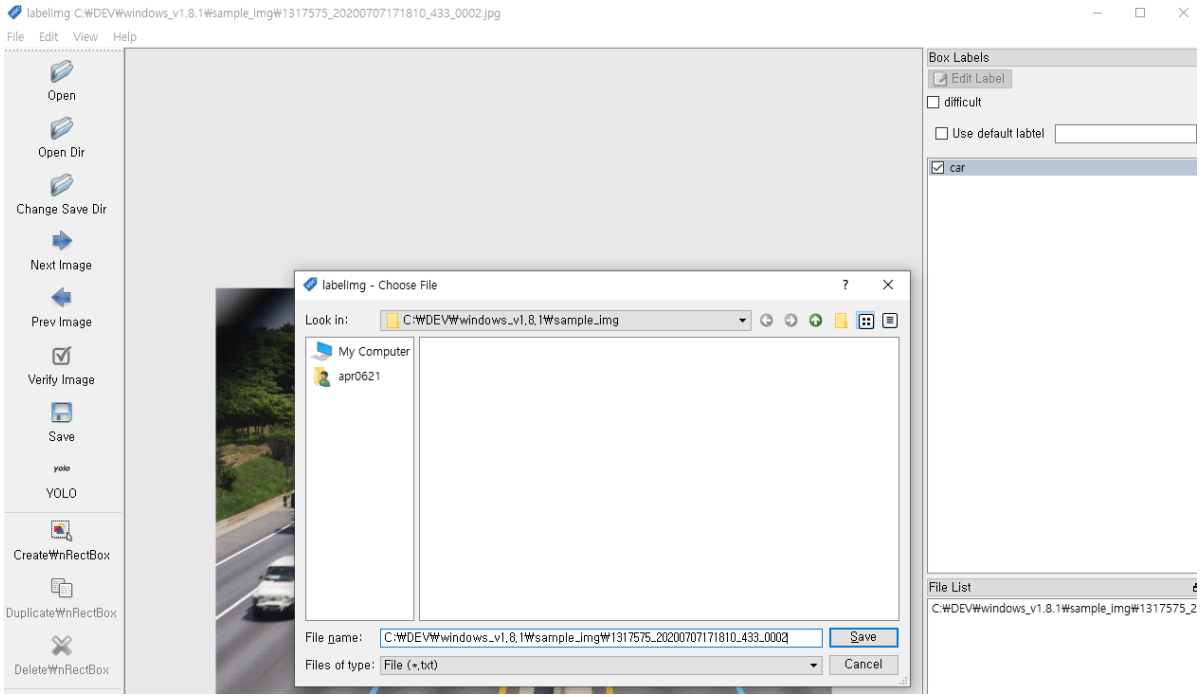
4. 단축키

- …



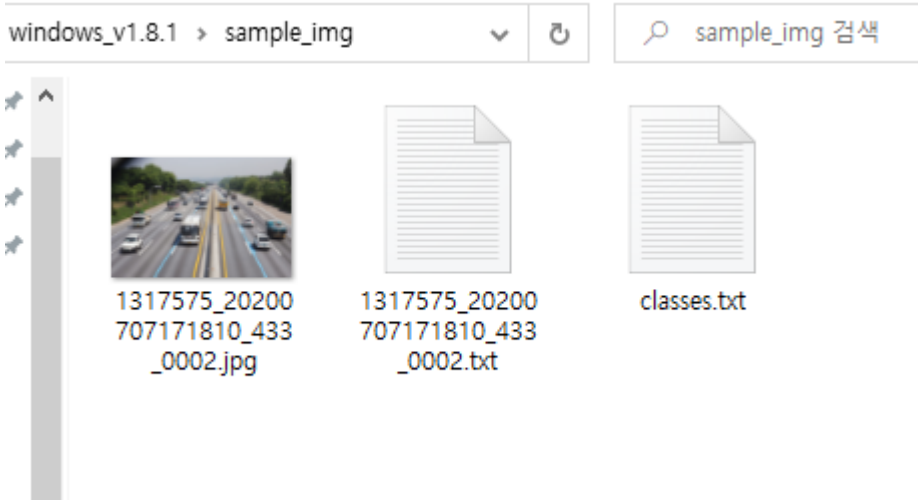
4. 단축키

- 정상적으로 진행되었다면 이미지에 Box 영역이 표시되고 우측에 해당하는 label이 보일 것이다. 라벨링을 완료했다면 Ctrl+S를 통해 저장해주면 된다.정상적으로 진행되었다면 이미지에 Box 영역이 표시되고 우측에 해당하는 label이 보일 것이다. 라벨링을 완료했다면 Ctrl+S를 통해 저장해주면 된다.



4. 단축키

- 라벨링된 파일을 살펴보면 다음과 같이 class 번호와 좌표가 저장된 것을 확인할 수 있다.

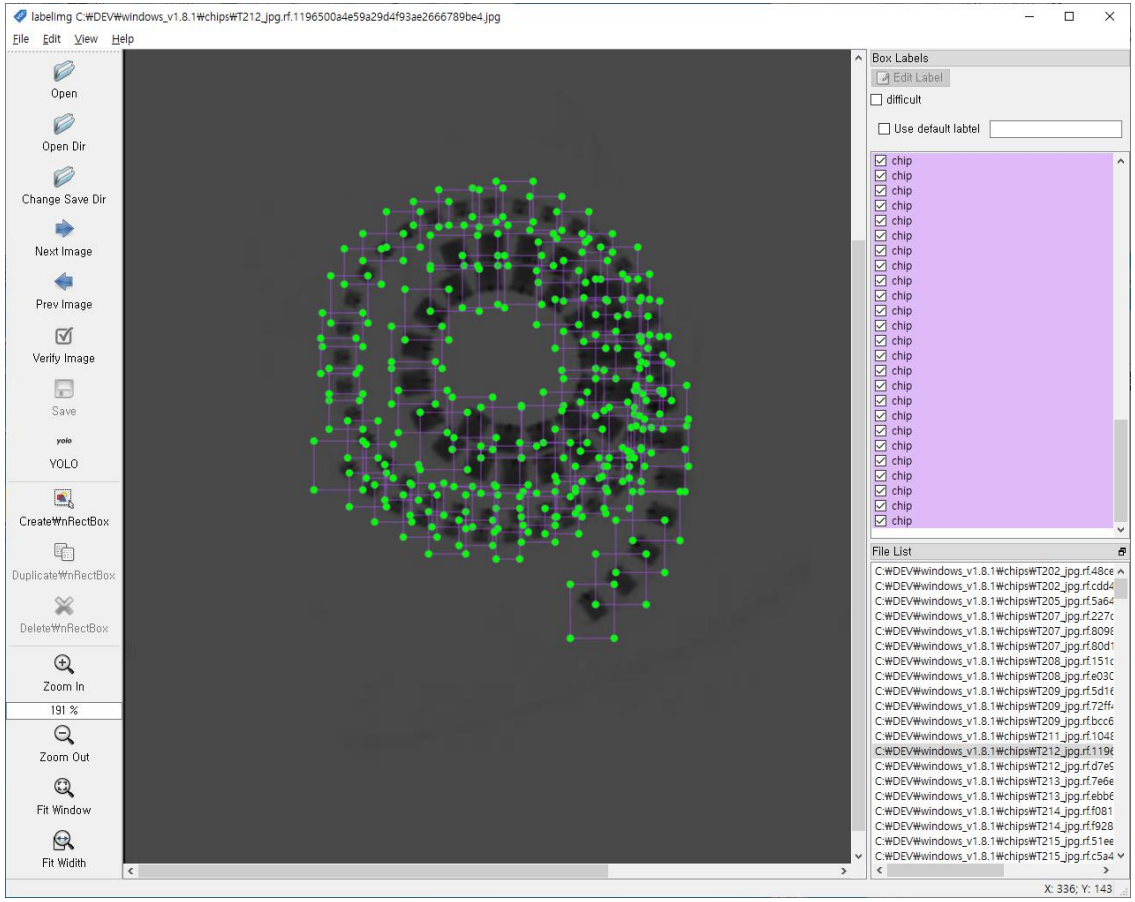



```
4 0.841000 0.712144 0.108000 0.167916
```

2. YOLO (You Only Look Once)

1. 라벨 결과 화면

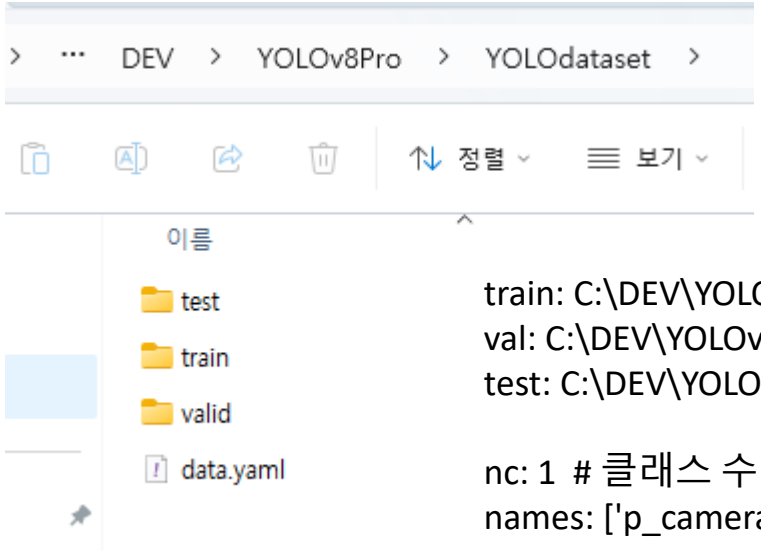
- 최대한 박스 작업을 해야 함



2. YOLO (You Only Look Once)

YOLOv8 커스텀 데이터 학습하기

- YOLOv8 은 사전학습 모델종류와 데이터 위치를 나타내는 yaml 파일만 바꾸어 주면, 동일한 Python API 로 Object Detection 과 Image Segmentation 구현이 가능합니다.




```
train: C:\DEV\YOLOv8Pro\YOLOdataset\train\images
val: C:\DEV\YOLOv8Pro\YOLOdataset\valid\images
test: C:\DEV\YOLOv8Pro\YOLOdataset\test\images

nc: 1  # 클래스 수
names: ['p_camera']  # 클래스 이름
```

2. YOLO (You Only Look Once)

YOLOv8 커스텀 데이터 학습하기

data architecture
yaml 파일에서 설정한 디렉토리 위치대로 구조를 설정하고, train/valid/test에는 각각 images, labels folder를 생성한다.

각 images, labels 파일에는 image file과 label(annotation text) file을 위치하면 되며, 여기서 훈련 시 train/valid 비율은 7.5:2.5정도로 설정했다.

```
C:\DEV\YOLOv8Pro\YOLOdataset>tree
폴더 PATH의 목록입니다.
볼륨 일련 번호는 FC5F-C327입니다.
C:.
 ├─test
 │   ├─images
 │   └─labels
 ├─train
 │   ├─images
 │   └─labels
 └─valid
     ├─images
     └─labels
```

2. YOLO (You Only Look Once)

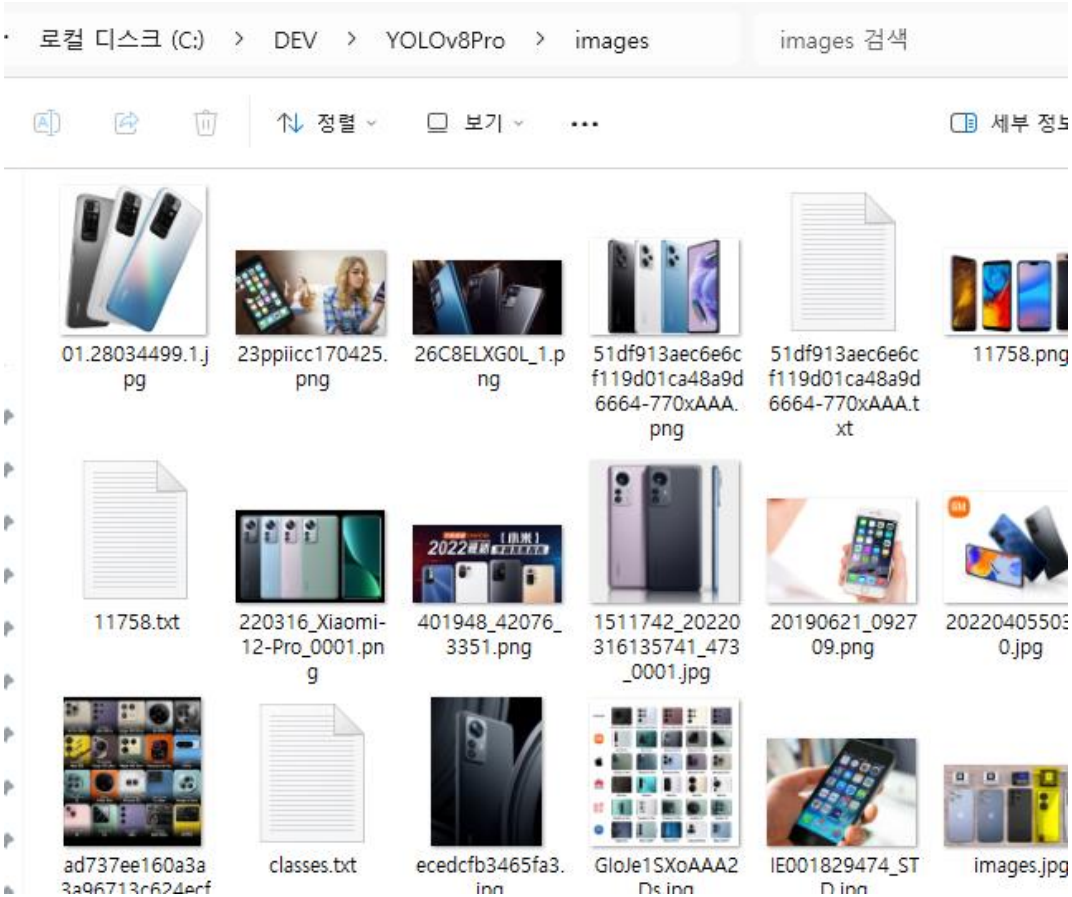
YOLOv8 커스텀 데이터 학습하기

```
C:\DEV\YOLOv8Pro\YOLOdataset>tree /f
폴더 PATH의 목록입니다.
볼륨 일련 번호는 FC5F-C327입니다.
C:.
│  data.yaml
│
├─test
│  ├─images
│  │     11758.png
│  │     51df913aec6e6cf119d01ca48a9d6664-770xAAA.png
│  │
│  └─labels
│        11758.txt
│        51df913aec6e6cf119d01ca48a9d6664-770xAAA.txt
│
├─train
│  │  labels.cache
│  │
│  ├─images
│  │     11758.png
│  │     51df913aec6e6cf119d01ca48a9d6664-770xAAA.png
│  │
│  └─labels
│        11758.txt
│        51df913aec6e6cf119d01ca48a9d6664-770xAAA.txt
│
└─valid
   │  labels.cache
   │
   ├─images
   │     11758.png
   │     51df913aec6e6cf119d01ca48a9d6664-770xAAA.png
   │
   └─labels
         11758.txt
         51df913aec6e6cf119d01ca48a9d6664-770xAAA.txt


C:\DEV\YOLOv8Pro\YOLOdataset>
```

2. YOLO (You Only Look Once)

YOLOv8 커스텀 데이터 만들기

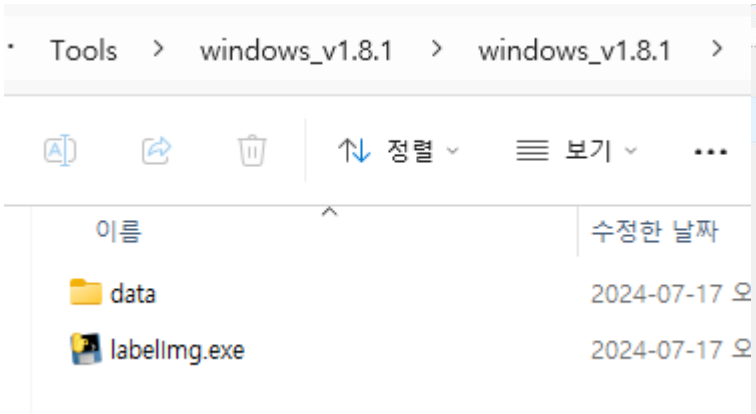
데이터를 수집합니다.

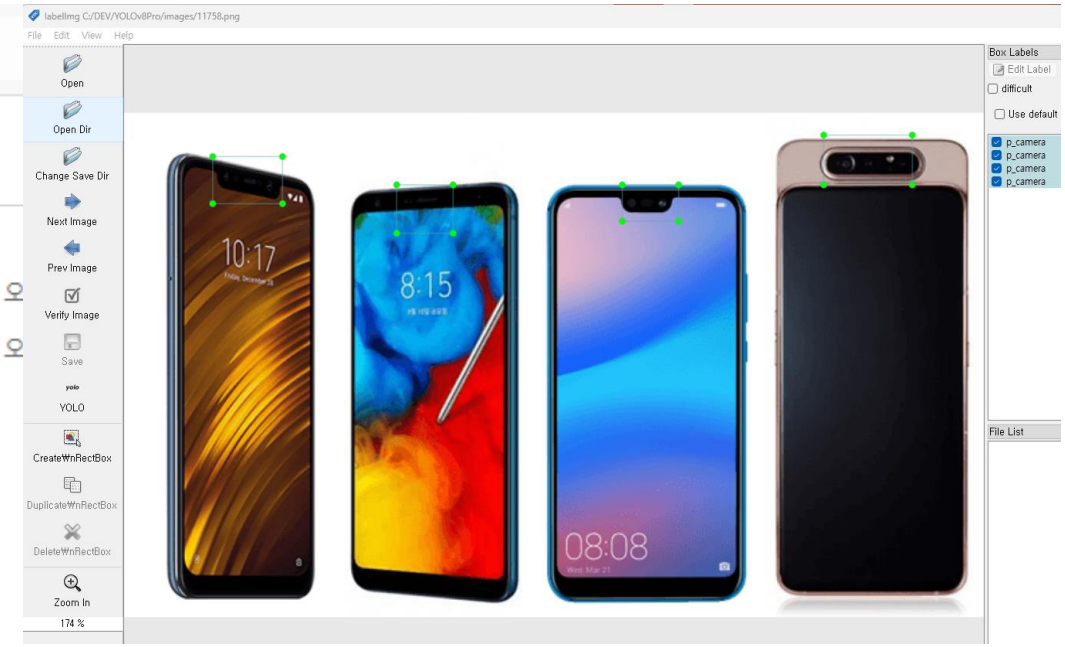


2. YOLO (You Only Look Once)

YOLOv8 커스텀 데이터 만들기

labelImg를 이용해 라벨링을 합니다.

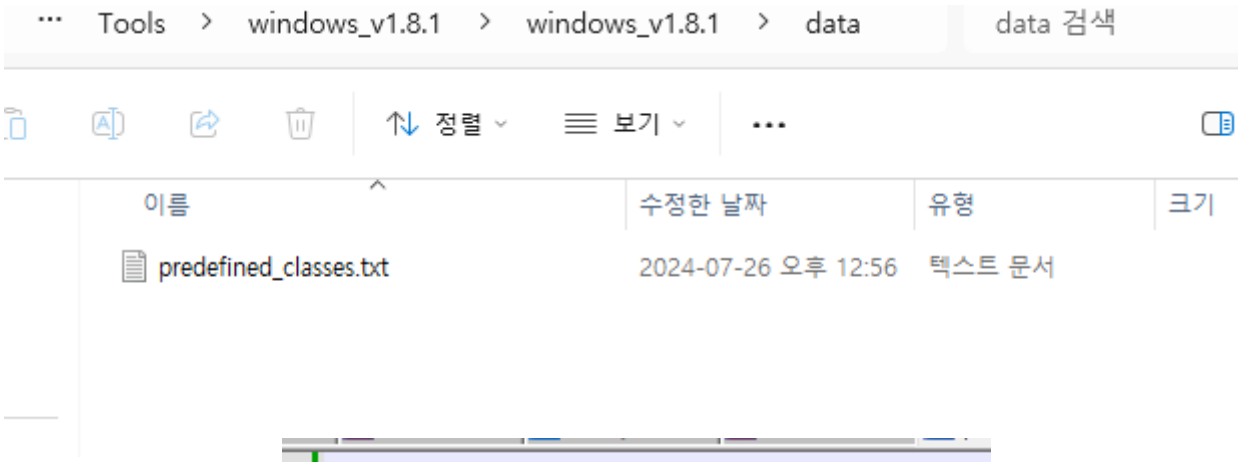


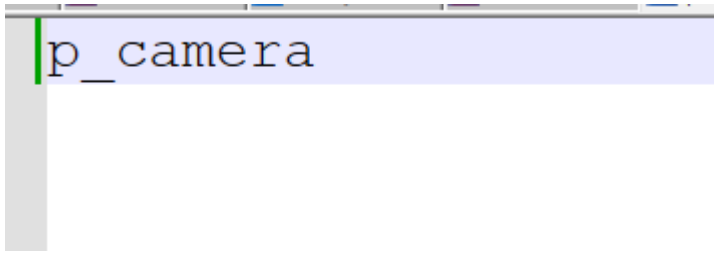


2. YOLO (You Only Look Once)

YOLOv8 커스텀 데이터 만들기

labelImg를 이용해 라벨링 시 predefined_classes.txt 파일 내용을 작업할 클래스명으로 초기화하시고 하셔야 합니다.





2. YOLO (You Only Look Once)

YOLOv8 커스텀 데이터 만들기

labelImg를 이용해 라벨링된 파일을 아키텍처에 맞게 배치합니다.

51df913aec6e6cf119d01ca48a9d6664-770xAAA.png

51df913aec6e6cf119d01ca48a9d6664-770xAAA.txt

11758.png

11758.txt

DEV > YOLOv8Pro > YOLOdataset >

정렬 보기

이름

test

train

valid

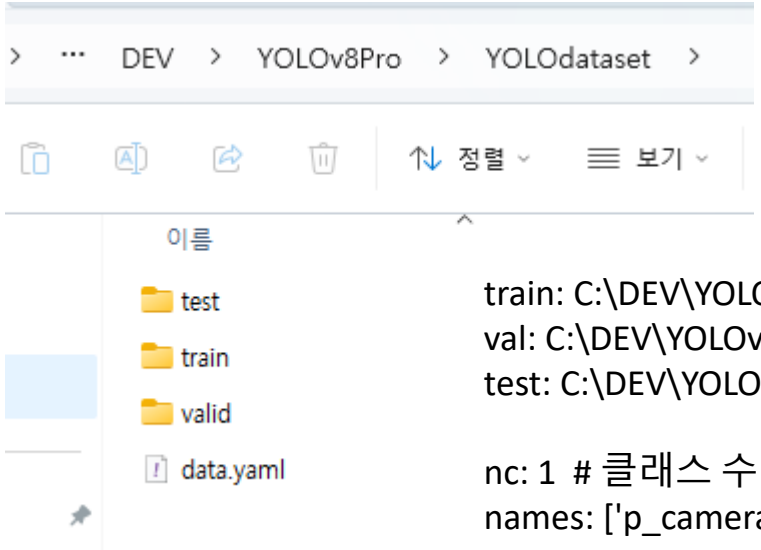
data.yaml

```
C:\DEV\YOLOv8Pro\YOLOdataset>tree /f
폴더 PATH의 목록입니다.
볼륨 일련 번호는 FC5F-C327입니다.
C:.
│  data.yaml
│
├─test
│  ├─images
│  │     11758.png
│  │     51df913aec6e6cf119d01ca48a9d6664-770xAAA.png
│  │
│  └─labels
│        11758.txt
│        51df913aec6e6cf119d01ca48a9d6664-770xAAA.txt
│
├─train
│  │  labels.cache
│  │
│  ├─images
│  │     11758.png
│  │     51df913aec6e6cf119d01ca48a9d6664-770xAAA.png
│  │
│  └─labels
│        11758.txt
│        51df913aec6e6cf119d01ca48a9d6664-770xAAA.txt
│
└─valid
   │  labels.cache
   │
   ├─images
   │     11758.png
   │     51df913aec6e6cf119d01ca48a9d6664-770xAAA.png
   │
   └─labels
         11758.txt
         51df913aec6e6cf119d01ca48a9d6664-770xAAA.txt

C:\DEV\YOLOv8Pro\YOLOdataset>
```

2. YOLO (You Only Look Once)

YOLOv8 커스텀 데이터 학습하기

- 데이터 위치를 나타내는 yaml 파일을 바꿔 줍니다.




```
train: C:\DEV\YOLOv8Pro\YOLOdataset\train\images
val: C:\DEV\YOLOv8Pro\YOLOdataset\valid\images
test: C:\DEV\YOLOv8Pro\YOLOdataset\test\images

nc: 1  # 클래스 수
names: ['p_camera']  # 클래스 이름
```

2. YOLO (You Only Look Once)

YOLOv8 커스텀 데이터 학습하기

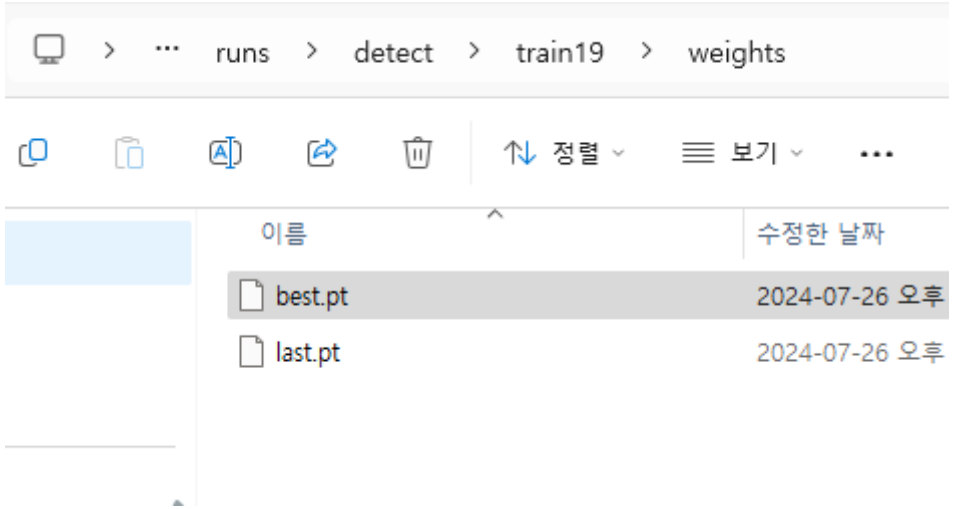
- 준비가 되면 학습을 진행합니다.

```
Microsoft Windows [Version 10.0.22631.3880]
(c) Microsoft Corporation. All rights reserved.

(py3_10_yolov8) C:\DEV\YOLOv8Pro>yolo task=detect mode=train model=yolov8s.pt data=YOLOdataset\data.yaml
epochs=100 imgsz=640

Ultralytics YOLOv8.2.66 🚀 Python-3.10.11 torch-2.3.1+cpu CPU (13th Gen Intel Core(TM) i7-13700)
engine\trainer: task=detect, mode=train, model=yolov8s.pt, data=YOLOdataset\data.yaml, epochs=100, time=None, patience=100, batch=16, imgsz=640,
save=True, save_period=-1, cache=False, device=None, workers=8, project=None, name=train19, exist_ok=False, pretrained=True, optimizer=auto, verbose=True,
seed=0, deterministic=True, single_cls=False, rect=False, cos_lr=False, close_mosaic=10, resume=False, amp=True, fraction=1.0, profile=False, freeze=None,
multi_scale=False, overlap_mask=True, mask_ratio=4, dropout=0.0, val=True, split=val, save_json=False, save_hybrid=False, conf=None, iou=0.7, max_det=300,
half=False, dnn=False, plots=True, source=None, vid_stride=1, stream_buffer=False, visualize=False, augment=False, agnostic_nms=False, classes=None,
retina_masks=False, embed=None, show=False, save_frames=False, save_txt=False, save_conf=False, save_crop=False, show_labels=True, show_conf=True,
show_boxes=True, line_width=None, format=torchscript, keras=False, optimize=False, int8=False, dynamic=False, simplify=False, opset=None, workspace=4,
nms=False, lr0=0.01, lrf=0.01, momentum=0.937, weight_decay=0.0005, warmup_epochs=3.0, warmup_momentum=0.8, warmup_bias_lr=0.1, box=7.5, cls=0.5,
dfl=1.5, pose=12.0, kobj=1.0, label_smoothing=0.0, nbs=64, hsv_h=0.015, hsv_s=0.7, hsv_v=0.4, degrees=0.0, translate=0.1, scale=0.5, shear=0.0, perspective=0.0,
flipud=0.0, fliplr=0.5, bgr=0.0, mosaic=1.0, mixup=0.0, copy_paste=0.0, auto_augment=randaugment, erasing=0.4, crop_fraction=1.0, cfg=None,
tracker=botsort.yaml, save_dir=runs\detect\train19
Overriding model.yaml nc=80 with nc=3
```

2. YOLO (You Only Look Once)

YOLOv8 커스텀 데이터 학습하기

학습 실행 결과는 C:\DEV\YOLOv8Pro\runs\detect\train19\weights에 best.pt, last.pt로 저장될 것이다.

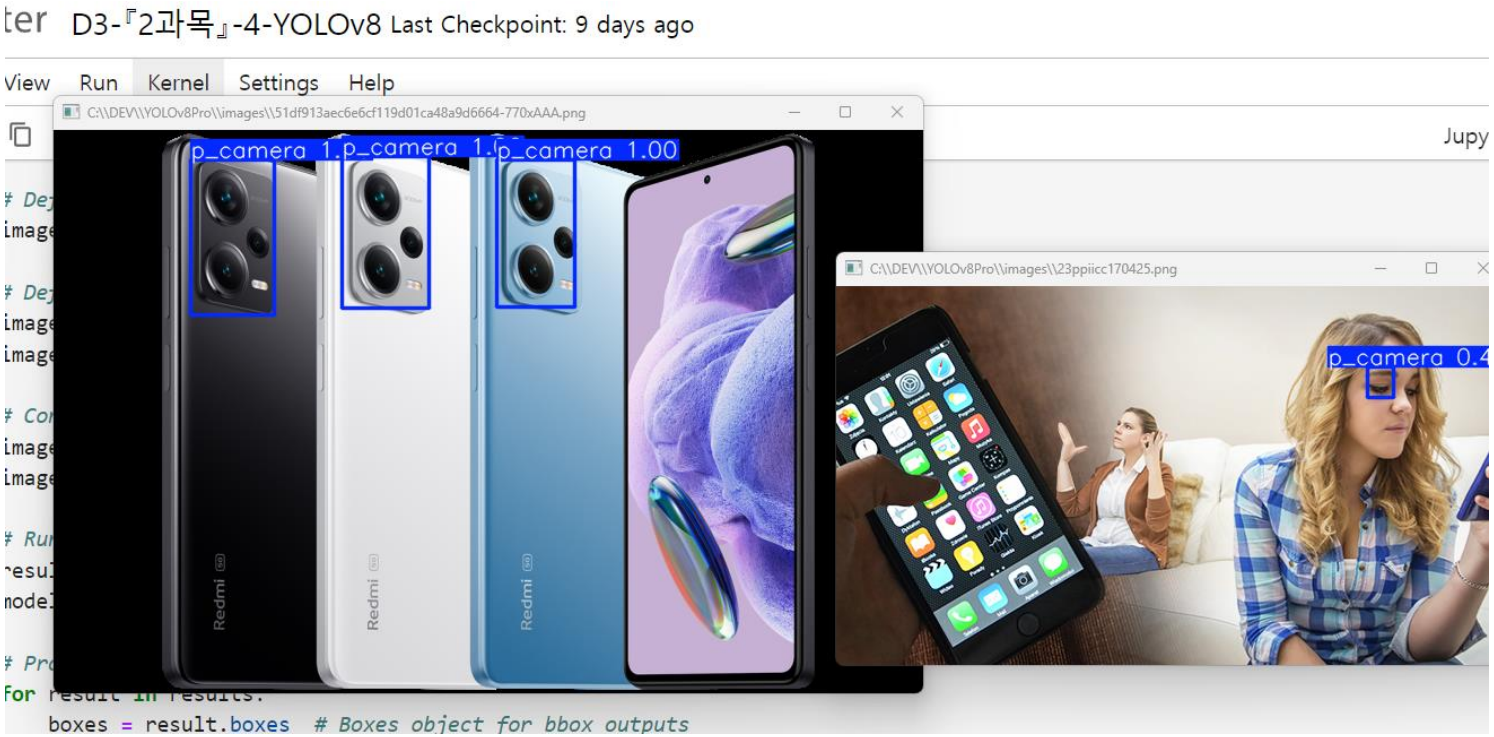


2. YOLO (You Only Look Once)

YOLOv8 커스텀 데이터 학습 모델 사용하기

```
from ultralytics import YOLO
import cv2
# Load a model
model = YOLO('C:\\DEV\\YOLOv8Pro\\runs\\detect\\train19\\weights\\best.pt')  # pretrained YOLOv8n model

model.predict(source="0", show=True, stream=True, classes=0)  # [0, 3, 5] for multiple classes


# Define image file path
image_folder = 'C:\\DEV\\YOLOv8Pro\\images\\'

# Define image file names
image_file_1 = '51df913aec6e6cf119d01ca48a9d6664-770xAAA.png'
image_file_2 = '23ppiicc170425.png'

# Construct full image file paths
image_path_1 = image_folder + image_file_1
image_path_2 = image_folder + image_file_2

# Run batched inference on a list of images
results = model([image_path_1, image_path_2])  # return a list of Results objects
model.predict(source="0", show=True, stream=True, classes=0)

# Process results list
for result in results:
    boxes = result.boxes  # Boxes object for bbox outputs
    masks = result.masks  # Masks object for segmentation masks outputs
    keypoints = result.keypoints  # Keypoints object for pose outputs
    probs = result.probs  # Class probabilities for classification outputs

    # Print box coordinates
    print("Box coordinates:")
    for box in boxes:
        print(box)

    # Display result with boxes
    result.plot()
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

2. YOLO (You Only Look Once)

YOLOv8 커스텀 데이터 학습 모델 사용하기

결과